

Universidad del Valle de Guatemala

Programación Orientada a Objetos

Segundo Ciclo, 2026

Catedrático: Kevin Antonio Velasquez Aguilar



Ejercicio 4 – Herencia
RentaMovil

Obed Natanael Tot Paiz
Carné:26971
Fecha de entrega: 27/9/2026

• Análisis

1. Requisitos funcionales

El programa permitirá administrar los vehículos de RentaMovil mediante un menú de consola. Deberá realizar las siguientes operaciones:

- Registrar automóviles, motocicletas y camionetas de carga.
- Consultar la flota y mostrar las características y disponibilidad de cada vehículo.
- Buscar vehículos mediante su placa.
- Cotizar un alquiler indicando una cantidad de días.
- Mostrar el costo antes de solicitar la confirmación del alquiler.
- Confirmar el alquiler de un vehículo disponible y acumular el ingreso correspondiente.
- Registrar devoluciones para que los vehículos vuelvan a estar disponibles.
- Mostrar un reporte con las cantidades de vehículos registrados, disponibles y alquilados por categoría, además de los ingresos acumulados.
- Mantener el menú activo hasta que el usuario seleccione salir.

Las cotizaciones podrán realizarse incluso cuando el vehículo esté alquilado. Cotizar o cancelar una operación no modificará la disponibilidad ni los ingresos. Una devolución cambiará la disponibilidad, pero no generará otro cobro ni descontará ingresos.

2. Información que conservará el sistema

Todos los vehículos tendrán placa, marca, modelo, tarifa diaria y disponibilidad y la empresa conservará una colección de vehículos y el total de ingresos recibidos por alquileres confirmados.

Categoría	Información específica
Automóvil	Cantidad de pasajeros y si su transmisión es automática.
Motocicleta	Cilindraje en centímetros cúbicos.
Camioneta de carga	Capacidad máxima de carga en toneladas, incluyendo decimales.

3. Reglas para calcular los cobros

El recargo de la motocicleta se aplicará una sola vez por alquiler, en las camionetas, el recargo dependerá de su capacidad máxima, aunque el cliente transporte menos carga. Los montos se manejarán como double y se mostrarán con dos decimales.

Vehículo	Costo total
Automóvil manual	$\text{tarifaDiaria} \times \text{dias}$
Automóvil automático	$(\text{tarifaDiaria} + 50) \times \text{dias}$
Motocicleta de hasta 250 cc	$\text{tarifaDiaria} \times \text{dias}$
Motocicleta de más de 250 cc	$(\text{tarifaDiaria} \times \text{dias}) + 75$
Camioneta de carga	$(\text{tarifaDiaria} + 100 \times \text{capacidadToneladas}) \times \text{dias}$

4. Clases y responsabilidades

Clase	Responsabilidad
Vehiculo	Reunir los atributos y comportamientos comunes de todos los vehículos. Será una clase abstracta.
Automovil	Representar automóviles y calcular el recargo por transmisión automática.
Motocicleta	Representar motocicletas y aplicar el recargo según el cilindraje.
CamionetaCarga	Representar camionetas y calcular el recargo según su capacidad máxima.
RentaMovil	Administrar la flota, validar las operaciones, acumular ingresos y generar reportes.
Principal	Contener main(), mostrar el menú, solicitar información y presentar resultados.

5. Atributos y métodos de las clases

Todos los atributos serán private para proteger el estado de los objetos.

- **Clase Vehiculo**

Atributo	Tipo	Propósito
placa	String	Identificar el vehículo.
marca	String	Guardar su marca.
modelo	String	Guardar su modelo.
tarifaDiaria	double	Establecer el precio base por día.
disponible	boolean	Indicar si puede alquilarse.

- **Metodos**

La placa se conservará como identificador y no tendrá un método para modificarla y la disponibilidad cambiará mediante alquilar () y devolver().

Método	Visibilidad	Resultado y propósito
getPlaca()	public	Devuelve la placa como String.
getMarca()	public	Devuelve la marca como String.
getModelo()	Public	Devuelve el modelo como String.
getTarifaDiaria()	Public	Devuelve la tarifa como double.
isDisponible()	Public	Devuelve la disponibilidad como boolean.
calcularCosto(int dias)	Public	Valida los días y devuelve el costo base como double.
alquilar()	Public	Valida los días y devuelve el costo base como double.

devolver()	Public	Comprueba que esté alquilado y lo marca como disponible. Devuelve void.
getCategoria()	public abstract	Cada clase hija devolverá su categoría como String.
toString()	public	Devuelve un String con la información común del vehículo.

Clases Automovil, Motocicleta y CamionetaCarga, estas clases heredarán de Vehiculo y van a tener ciertos atributos privados.

Clase	Atributo	Tipo	Propósito
Automovil	cantidadPasajeros	Int	Indicar cuántos pasajeros admite.
Automovil	automatico	Boolean	Indicar si tiene transmisión automática.
Motocicleta	cilindraje	Int	Guardar el cilindraje en cc.
CamionetaCarga	capacidadToneladas	double	Guardar la capacidad máxima de carga.

- Las tres clases tendrán métodos públicos

Método	Propósito
calcularCosto(int dias): double	Obtener el costo base mediante super.calcularCosto(dias) y agregar el recargo correspondiente.
getCategoria(): String	Devolver el nombre de la categoría.
toString(): String	Añadir las características particulares a la información común.

- Clase RentaMovil

Atributo	Tipo	Propósito
vehiculos	ArrayList<Vehiculo>	Almacenar en una misma colección las tres categorías.
ingresosAcumulados	double	Guardar el dinero recibido por alquileres confirmados.

- **Métodos de esa clase**

Las cantidades del reporte se calcularán a partir de la colección, evitando mantener contadores separados que puedan quedar desactualizados.

Método	Visibilidad	Resultado y propósito
registrarVehiculo(Vehiculo vehiculo)	Public	Valida que no sea null ni tenga una placa repetida y lo agrega. Devuelve void.
buscarVehiculo(String placa)	Public	Devuelve el Vehiculo encontrado. Si no existe, genera un error de validación.
consultarFlota()	Public	Devuelve un String con los vehículos y sus características.
cotizar(String placa, int dias)	Public	Devuelve el costo como double, sin cambiar disponibilidad ni ingresos.
confirmarAlquiler(String placa, int dias)	Public	Valida la operación, registra el alquiler y acumula el ingreso. Devuelve el monto cobrado como double.
devolverVehiculo(String placa)	Public	Registra la devolución sin modificar los ingresos. Devuelve void.

getIngresosAcumulados()	Public	Devuelve el total de ingresos como double.
generarReporte()	Public	Devuelve un String con las cantidades generales, por categoría y los ingresos.
contarVehiculos(String categoria)	Private	Devuelve la cantidad registrada de esa categoría como int.
contarDisponibles(String categoria)	private	Devuelve la cantidad disponible de esa categoría como int.
contarAlquilados(String categoria)	private	Devuelve la cantidad alquilada de esa categoría como int.

- **Clase Principal**

Tendrá los atributos entrada, de tipo Scanner, y empresa, de tipo RentaMovil. Ambos serán private static.

Método	Visibilidad	Resultado y propósito
main(String[] args)	public static	Inicia el programa y controla su ejecución. Devuelve void.
mostrarMenu()	private static	Muestra las opciones. Devuelve void.
ejecutarOpcion(int opcion)	private static	Ejecuta la operación elegida y devuelve un boolean que indica si debe continuar el menú.
cargarDatosIniciales()	private static	Registra los seis vehículos de demostración. Devuelve void.
leerTexto(String mensaje)	private static	Solicita y devuelve un String no vacío.

leerEntero(String mensaje)	private static	Solicita y devuelve un int, controlando formatos incorrectos.
leerDecimal(String mensaje)	private static	Solicita y devuelve un double, controlando formatos incorrectos.
leerConfirmacion(String mensaje)	private static	Acepta una respuesta de sí o no y devuelve un boolean.

6. Constructores y valores iniciales

Los constructores recibirán los datos necesarios para crear objetos válidos:

Clase	Parámetros del constructor	Visibilidad
Vehiculo	String placa, String marca, String modelo, double tarifaDiaria	protected
Automovil	String placa, String marca, String modelo, double tarifaDiaria, int cantidadPasajeros, boolean automatico	public
Motocicleta	String placa, String marca, String modelo, double tarifaDiaria, int cilindraje	Public
CamionetaCarga	String placa, String marca, String modelo, double tarifaDiaria, double capacidadToneladas	Public
RentaMovil	Sin parámetros.	Public

Los constructores de las clases hijas utilizarán super(...) para inicializar los atributos comunes y cada vehículo comenzará con disponible = true.

El constructor de RentaMovil creará un ArrayList<Vehiculo> vacío e iniciará los ingresos en 0.0. y mostrara después del principal.

- Dos automóviles: uno manual y otro automático.
- Dos motocicletas: una de 250 cc y otra de más de 250 cc.
- Dos camionetas con capacidades diferentes, incluyendo una de 1.5 toneladas.

Todos tendrán placas distintas y datos válidos.

7. Validaciones y manejo de errores

Antes de registrar o modificar el estado del sistema se comprobará:

- La placa, marca y modelo no podrán ser null, estar vacíos ni contener solamente espacios.
- Las placas se guardarán sin espacios al inicio o al final y en mayúsculas para detectar duplicados.
- La tarifa diaria, cantidad de pasajeros, cilindraje y capacidad de carga deberán ser mayores que cero.
- Los valores decimales deberán ser finitos.
- Los días deberán ser enteros positivos.
- El vehículo deberá existir para cotizarlo, alquilarlo o devolverlo.
- Un vehículo ocupado no podrá volver a alquilarse.
- Un vehículo disponible no podrá registrarse como devuelto.
- Cancelar una confirmación conservará el estado anterior.

Los errores de formato numérico se manejarán en los métodos de lectura mediante try-catch, solicitando nuevamente el dato y las operaciones inválidas generarán `IllegalArgumentException` o `IllegalStateException` y estas serán capturadas en `Principal` para mostrar un mensaje y continuar. Y las validaciones y el cálculo del monto se realizarán antes de cambiar la disponibilidad o los ingresos.

8. Colaboración de los objetos durante un alquiler

El proceso es este:

1. `Principal` solicitará la placa y la cantidad de días.
2. `RentaMovil` localizará el vehículo y comprobará su disponibilidad.
3. El vehículo calculará el costo utilizando el método correspondiente a su categoría.
4. `Principal` mostrará sus características, disponibilidad y monto total, y solicitará confirmación.

5. Si el usuario acepta, Principal llamará a confirmarAlquiler(). Este método volverá a validar la operación, marcará el vehículo como alquilado y sumará el cobro a los ingresos.
6. Si el usuario cancela, no se ejecutará la confirmación y los datos permanecerán iguales.
7. Para una cotización independiente se permitirá consultar cualquier vehículo existente, aunque esté ocupado.

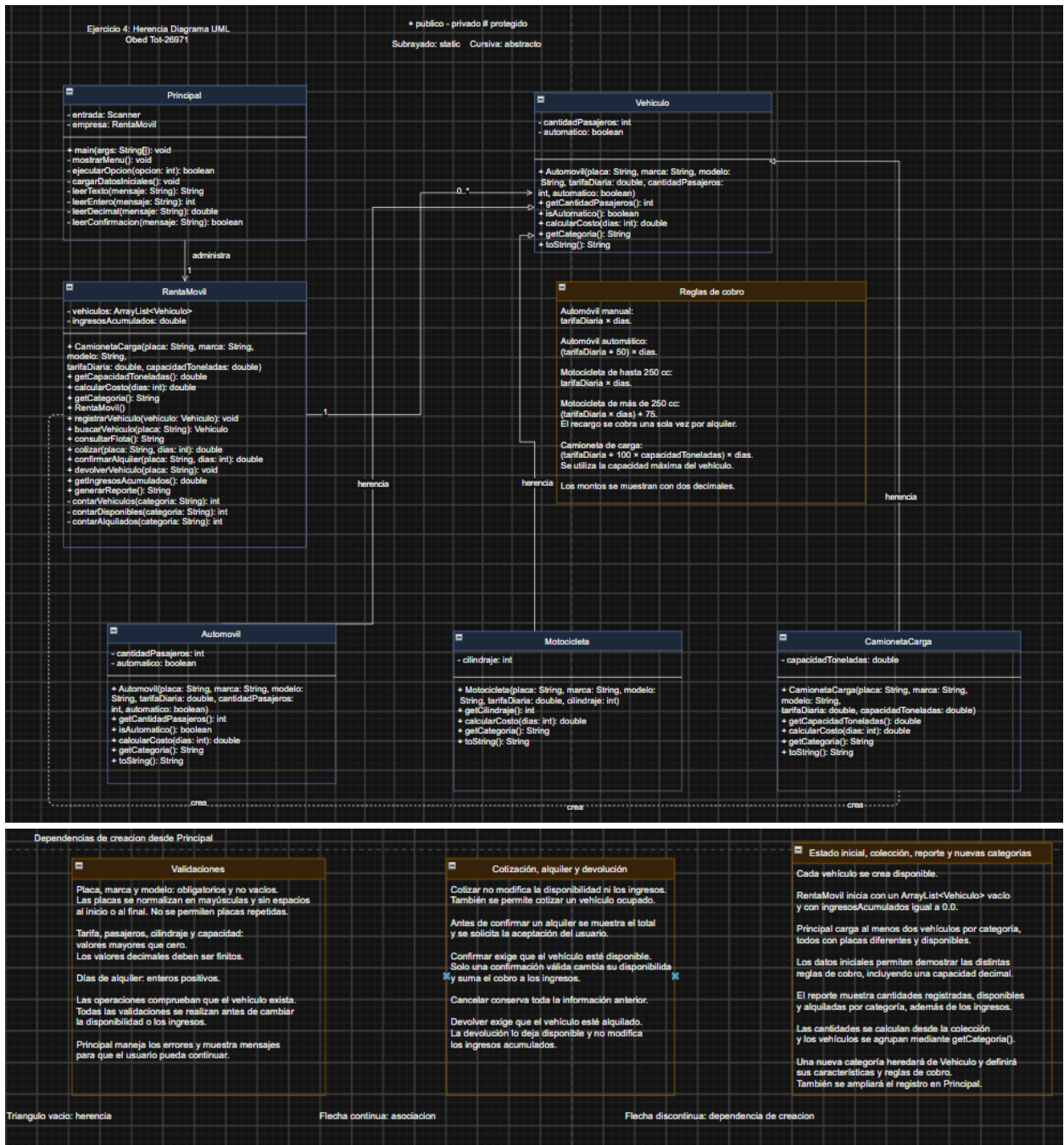
9. Justificación de la herencia y futuras categorías

Automovil, Motocicleta y CamionetaCarga heredarán de Vehiculo porque comparten datos y comportamientos, así se evitará repetir la placa, marca, modelo, tarifa y control de disponibilidad. Cada clase hija será responsable de sus características y reglas de cobro; mediante el polimorfismo, RentaMovil podrá llamar a calcularCosto(dias) sobre un objeto de tipo Vehiculo, y Java ejecutará la versión correspondiente a su categoría.

Para agregar una nueva categoría se creará otra clase que herede de Vehiculo, defina sus atributos y sobrescriba los métodos necesarios. También se ampliará la opción de registro en Principal. La colección y las operaciones generales de cotización, alquiler y devolución podrán mantenerse y el reporte agrupará los vehículos mediante getCategoria() para reconocer las nuevas categorías.

• Diseño

Diagrama UML



• Programa Evidencias.

```
Problems 4 Output Debug Console Terminal Ports
Se cargaron seis vehiculos disponibles. Ingresos iniciales: Q0.00.

----- Menu -----
1. Registrar vehiculo
2. Consultar flota
3. Cotizar alquiler
4. Confirmar alquiler
5. Registrar devolucion
6. Mostrar reporte general
7. Salir
Seleccione una opcion: █
```

Evidencia 1. Inicio del programa

El programa mostró el mensaje de carga inicial y las siete opciones del menú.

```
Problems 4 Output Debug Console Terminal Ports
2. Consultar flota
3. Cotizar alquiler
4. Confirmar alquiler
5. Registrar devolucion
6. Mostrar reporte general
7. Salir
Seleccione una opcion: 1
1. Automovil
2. Motocicleta
3. Camioneta de carga
Tipo de vehiculo: █
```

Evidencia 2. Selección del tipo de vehículo

Registrar vehículo, mostrar las categorías disponibles, aparecieron las opciones Automóvil, Motocicleta y Camioneta de carga.

```
Problems 4 Output Debug Console Terminal Ports
Cilindraje en cc (mayor que 0): 3600
Vehiculo registrado y disponible.

----- Menu -----
1. Registrar vehiculo
2. Consultar flota
3. Cotizar alquiler
4. Confirmar alquiler
5. Registrar devolucion
6. Mostrar reporte general
7. Salir
```

Evidencia 3. Registro de un vehículo

Después de ingresar los datos, el programa mostró el mensaje “Vehículo registrado y disponible” y regresó al menú principal.

```
Problems Output Debug Console Terminal Ports
Seleccione una opcion: 2
FLOTA DE RENTAMOVIL
Placa: A001 | Marca: Toyota | Modelo: Yaris | Categoria: Automovil | Tarifa diaria: Q150.00 | Estado: Disponible | Pasajeros: 5 | Transmision: Manual
Placa: A002 | Marca: Honda | Modelo: Civic | Categoria: Automovil | Tarifa diaria: Q150.00 | Estado: Disponible | Pasajeros: 5 | Transmision: Automatica
Placa: M001 | Marca: Yamaha | Modelo: FZ | Categoria: Motocicleta | Tarifa diaria: Q100.00 | Estado: Disponible | Cilindraje: 250 cc
Placa: M002 | Marca: Honda | Modelo: CB | Categoria: Motocicleta | Tarifa diaria: Q100.00 | Estado: Disponible | Cilindraje: 300 cc
Placa: C001 | Marca: Toyota | Modelo: Hilux | Categoria: Camioneta de carga | Tarifa diaria: Q200.00 | Estado: Disponible | Capacidad maxima: 1.5 toneladas
Placa: C002 | Marca: Isuzu | Modelo: D-Max | Categoria: Camioneta de carga | Tarifa diaria: Q250.00 | Estado: Disponible | Capacidad maxima: 2.0 toneladas
Placa: C003 | Marca: IZUZU | Modelo: CAMIONETA | Categoria: Motocicleta | Tarifa diaria: Q150.00 | Estado: Disponible | Cilindraje: 3600 cc
```

Evidencia 4. Consulta de la flota

La consulta mostró los siete vehículos registrados, incluyendo el nuevo vehículo con placa C003. Se presentaron sus placas, marcas, modelos, categorías, tarifas diarias, disponibilidad y características particulares.

```
Problems Output Debug Console Terminal Ports
7. Salir
Seleccione una opcion: 4
Placa del vehiculo: C002
Dias de alquiler (entero mayor que 0): 5
Placa: C002 | Marca: Isuzu | Modelo: D-Max | Categoria: Camioneta de carga | Tarifa diaria: Q250.00 | Estado: Disponible | Capacidad maxima: 2.0 toneladas
Total por 5 dia(s): Q2250.00
Confirma el alquiler? (s/n): S
Alquiler confirmado. Cobro registrado: Q2250.00
```

Evidencia 5. Confirmación del alquiler

Se alquiló la camioneta C002 durante cinco días. Su tarifa diaria de Q250.00 y su capacidad de dos toneladas produjeron un total de Q2250.00 y el programa mostró el monto antes de confirmar y registró el cobro al responder “s”.

```
Problems Output Debug Console Terminal Ports
3. Cotizar alquiler
4. Confirmar alquiler
5. Registrar devolucion
6. Mostrar reporte general
7. Salir
Seleccione una opcion: 5
Placa del vehiculo a devolver: C002
Devolucion registrada. Vehiculo disponible e ingresos sin cambios.
```

Evidencia 6. Devolución del vehículo

Se registró la devolución del vehículo C002 y el programa confirmó que quedó disponible nuevamente y que los ingresos se conservaron sin cambios.

```
Problems 4 Output Debug Console Terminal Ports

Disponibles: 7
Alquilados: 0
Automovil | Registrados: 2 | Disponibles: 2 | Alquilados: 0
Motocicleta | Registrados: 3 | Disponibles: 3 | Alquilados: 0
Camioneta de carga | Registrados: 2 | Disponibles: 2 | Alquilados: 0
Ingresos acumulados: Q2250.00
```

Evidencia 7. Reporte general

Después de la devolución, el reporte mostró siete vehículos disponibles y ninguno alquilado. Los ingresos acumulados están en Q2250.00 comprobando que la devolución no eliminó el cobro registrado.

```
5. Registrar devolucion
6. Mostrar reporte general
7. Salir
Seleccione una opcion: 7
Programa finalizado.
PS C:\P00\Ejercicio 4, Herencias>
```

Evidencia 8. Salida del programa

Al seleccionar la opción 7 el programa mostró Programa finalizado y terminó la ejecución correctamente.

- Enlace de GitHub.

<https://github.com/ntp90-cmd/Ejercicio-4-Herencia.git>